

Ứng dụng tác tử AI trong lập trình nhúng: không chỉ tăng năng suất mà còn mở rộng năng lực của kỹ sư

Vũ Trí Công, Nguyễn Trung Hiếu

Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, Việt Nam

Email: CongVT.B24CHDT005@stu.ptit.edu.vn, hieunt@ptit.edu.vn

Tóm tắt—Lợi ích của tác tử (agent) dùng mô hình ngôn ngữ lớn trong lập trình thường được đo bằng năng suất: số dòng mã, số giờ tiết kiệm. Bài báo cho rằng với lập trình nhúng, thước đo đó chưa đủ, và đề xuất đo thêm năng lực: tác tử có làm được những việc mà một kỹ sư làm tay thường bỏ, hoặc không làm được, hay không. Ba thước được đếm từ số ghi việc của tác tử EIDE trên ba việc thật: làm đủ bước (số phép đo và lần đổi chiều thực hiện), vào vùng mới (việc nằm ngoài vùng từng có công cụ, đi được tới bo thật), và tự khai điểm mù (điểm yếu của bộ kiểm được nêu trước rồi nổ đúng chỗ). Ba việc gồm nhân hệ điều hành thời gian thực tự viết trên STM32F469, robot hai bánh tự cân bằng trên ATmega328P làm hai lần, và lõi RISC-V trên FPGA Tang Nano 20K với 96 ô đo trên silicon. Về năng suất, phiên tác tử tốn 1,4–7,4 giờ của một người so với 31–32 ngày công ước lượng cho một đội, nhưng bài báo nêu bốn điều con số này không chứng minh. Về năng lực, tác tử làm đủ 96 ô đo kèm tổng kiểm, đi từ không có công cụ HDL tới bitstream chạy trên bo trong hai ngày, và tự khai hai điểm mù nổ đúng chỗ. Khi cùng một đề bài robot được làm lại với đề bài viết viết rõ, chiều bắt lỗi đối: lỗi tác tử do người bắt giảm $6 \rightarrow 1$, lỗi người do tác tử bắt tăng $0 \rightarrow 3$. Kết luận: mức lợi ti lệ với phần việc nằm trong máy tính, và chất lượng đề bài do người viết quyết định phần lớn kết quả.

Từ khóa—tác tử AI, mô hình ngôn ngữ lớn, lập trình nhúng, năng suất kỹ sư, cộng tác người-máy, FPGA, hệ điều hành thời gian thực.

I. GIỚI THIỆU

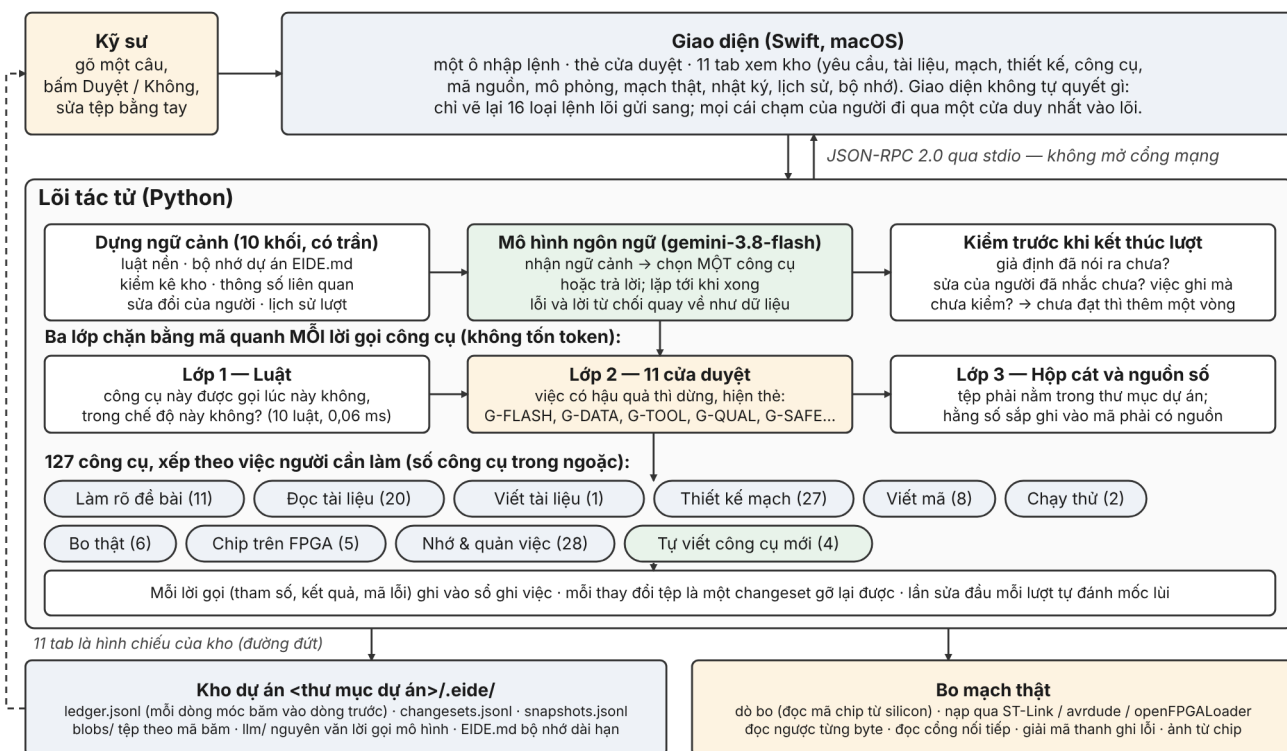
Quyết định số 21/2026/QĐ-TTg xếp trí tuệ nhân tạo và công nghệ bán dẫn vào danh mục công nghệ chiến lược [1]; lập trình nhúng nằm ở giao điểm hai hướng đó. Các nghiên cứu về mô hình ngôn ngữ lớn (LLM) cho lập trình nhúng đến

nay chủ yếu đo mô hình làm được gì trên bài tập: Englhardt và cộng sự [2] cho thấy LLM viết và sửa được mã cho vi điều khiển nhưng hay sai thông số và cần người kiểm; Li và cộng sự [3] thêm kỹ năng chuyên gia cho tác tử rồi đo tỷ lệ hoàn thành; Zhong và cộng sự [4] đo xu hướng mô hình tìm cách làm bộ kiểm xanh thay vì làm mã đúng. Khi nói về lợi ích cho kỹ sư, cách đo quen thuộc là năng suất: nhanh hơn bao nhiêu, rẻ hơn bao nhiêu.

Bài báo cho rằng với lập trình nhúng, năng suất là thước đo chưa đủ, vì hai lý do. Một là phần lớn thời gian của việc nhúng không nằm ở gõ mã mà ở đọc tài liệu, đối chiếu thông số, đo trên bo và tìm lỗi; tiết kiệm giờ gõ mã không nói gì về các phần đó. Hai là một kỹ sư làm tay có những việc *biết là nên làm nhưng thường bỏ* (đọc ngược chip sau mỗi lần nạp, đối chiếu lại toàn bộ điều kiện sau mỗi lần sửa, tính tổng kiểm cho từng ô đo) và những việc *không làm được vì chưa có công cụ*. Nếu tác tử làm được các việc đó thì nó mở rộng năng lực của kỹ sư, không chỉ rút ngắn thời gian; và điều ấy cần một thước đo riêng.

Bài báo đề xuất ba thước đo năng lực đếm được từ số ghi việc, rồi áp vào ba việc thật mà tác tử EIDE đã làm trên ba bo mạch khác nhau. Mục II giới thiệu tác tử: nó có gì và làm được gì; kiến trúc được so sánh với các kiến trúc khác ở [5], và kết quả từng việc được kể chi tiết ở [6]. Mã nguồn, số ghi việc và nhật ký của ba việc được công bố [7].

Đóng góp: (1) ba thước đo năng lực (làm đủ bước, vào vùng mới, tự khai điểm mù) và cách đếm chúng từ số ghi việc; (2) số đo năng suất và năng lực trên ba việc thật, trong đó hai việc được làm hai lần; (3) hai phát hiện: mức lợi ti lệ với phần việc nằm trong máy tính, và chất lượng đề bài do người viết quyết định kết quả nhiều hơn năng lực mô hình, thể hiện qua chiều bắt lỗi đối khi đề bài được viết lại.



Hình 1. Kiến trúc tác tử EIDE: giao diện chỉ vẽ lại thứ lỗi gửi; lỗi là một vòng lặp do mô hình điều khiển, mỗi lời gọi công cụ đi qua ba lớp chặn bằng mã; mọi việc ghi vào kho của dự án.

II. TÁC TỬ EIDE: CÓ GÌ VÀ LÀM ĐƯỢC GÌ

A. Kiến trúc

EIDE là một tác tử làm phần mềm nhúng *cùng* kỹ sư, không làm thay: hai bên ghi vào cùng một kho, và mọi việc tác tử làm đều để lại dấu đủ để người kiểm lại. Hình 1 vẽ các phần chính. Giao diện (Swift, macOS) có một ô nhập lệnh, các thẻ cửa duyệt và 11 tab xem kho; nó không tự quyết gì, chỉ vẽ lại thứ lỗi gửi sang. Lỗi (Python) là một vòng lặp do mô hình gemini-3.8-flash điều khiển: mô hình nhận ngữ cảnh dựng từ kho, chọn một trong 127 công cụ, nhận kết quả, rồi chọn tiếp cho tới khi trả lời; lỗi và lỗi từ chối quay về như dữ liệu nên mô hình đối hướng thay vì dừng. Mỗi lời gọi công cụ đi qua ba lớp chặn viết bằng mã, không tốn token: luật (công cụ này được gọi lúc này không), cửa duyệt (việc có hậu quả thì dừng, hiện thẻ chờ người bấm; có 11 cửa, như nạp chip, cài công cụ, đổi mức đo sau khi đã có kết quả), và hộp cát (tệp phải nằm trong thư mục dự án, hàng số sắp ghi vào mã phải có nguồn) [5].

Ba luật nền chi phối mọi thứ khác. Một, mỗi con số dùng để quyết định phải lần về được chỗ lấy ra: trang tài liệu của hãng, dòng tệp cấu hình, hay câu nói nguyên văn của người; mỗi thông số mang một bậc tin được (VÀNG, BẠC, NGƯỜI, CÀI ĐẶT, ĐỒNG) và đường nâng bậc chỉ người đi được. Hai, không có kết quả “đạt” nào mà thiếu bằng chứng: mức đo phải nêu trước khi chạy, và đổi mức đo sau khi thấy kết quả phải qua một cửa duyệt. Ba, mọi thay đổi gỡ lại được, kể cả thay đổi do người tự gõ; việc không gỡ lại được như nạp chip vẫn được ghi kèm lý do. Mọi lời gọi, tham số, kết quả, mã lỗi ghi vào sổ ghi việc có mốc bấm; mỗi thay đổi tệp là một changeset; nguyên văn từng lời gọi mô hình được lưu. Mọi con số trong bài này đều đếm từ các tệp đó.

B. Năng lực

Bảng I xếp 127 công cụ theo việc người cần làm. Hai năng lực được dùng nhiều trong bài: tác tử tự viết được công cụ mới kèm bộ kiểm và chỉ nạp khi bộ kiểm xanh (8/8 ca trong bộ thử); và công cụ *test.sensitivity* cố ý làm hỏng mã sản phẩm để xem bộ kiểm có dò không. Tác tử được thử ở bốn mức: 1 575 ca kiểm đơn vị; các ca đi qua giao thức thật; 76 ca kiểm thuộc 19 kịch bản chạy qua ứng dụng thật (68/68 ca đo được đạt); và việc thật trên bo thật, là mức duy nhất trả lời được câu “tác tử có gánh nổi một việc phức tạp không” [6].

BẢNG I
CÁC NHÓM VIỆC CỦA TÁC TỬ (SỐ CÔNG CỤ) VÀ ĐIỀU NÓ LÀM ĐƯỢC

Việc (số)	Tác tử làm được gì
Làm rõ đề bài (11)	hỏi gộp một cụm câu kèm lý do; ghi yêu cầu kèm câu nói nguyên văn; nêu 2–4 cách làm rồi để người chốt
Đọc tài liệu (20)	nạp PDF, Word, Excel, slide, mã nguồn; rút hình kèm chữ trong hình; lấy con số bằng mã theo trang, không để mô hình đọc hộ
Viết tài liệu (1)	dựng Word, PowerPoint, Excel, PDF từ nguồn Markdown trong kho, rồi mở lại tệp đếm đoạn, bảng, hình trước khi báo
Thiết kế mạch (27)	bản đồ mạch; cây khối nhiều cấp; sinh sơ đồ nguyên lý mở được bằng KiCad; không gán chân không có trong tài liệu
Viết mã (8)	đọc hiểu mã cũ trước khi sửa; dịch cho AVR, ARM Cortex-M, RISC-V; bản đồ bộ nhớ; đo xem bộ kiểm có đo gì không
Chạy thử trên máy (2)	nêu mức đo trước, chạy, rồi đối chiếu; bộ mức đo chấm, không phải chương trình
Bo thật (6)	dò bo; nạp qua ST-Link, avrdude; đọc ngược so từng byte; đọc cổng nối tiếp; giải mã thành ghi lỗi; đọc

Việc (số)	Tác tử làm được gì
	khung ảnh từ chip
Chip trên FPGA (5)	soát cú pháp Verilog; mô phỏng; tổng hợp; đặt-di dây và đo Fmax; đóng gói và nạp bitstream
Nhớ và quản việc (28)	kho; số ghi việc; gỡ lại; bản chốt; nhánh; chia việc lớn thành chặng có thứ mở ra xem được
Tự lo cho mình (4)	tìm công cụ; tự viết công cụ mới rồi tự kiểm trước khi dùng

C. Ba việc làm dữ liệu

Bảng II tóm tắt ba việc. Cả ba dùng cùng tác tử, cùng mô hình, cùng một người thao tác (tác giả thứ nhất). Hai việc đầu được làm hai lần: lần 1 trong vùng tác tử đã quen, lần 2 từ dự án trống sau khi vá các lỗi tìm được và viết lại đề bài cho rõ. Việc thứ ba cố ý chọn ngoài hẳn vùng từng làm: trước đó EIDE không có một dòng công cụ nào về HDL, tổng hợp Verilog, nạp FPGA hay dịch cho RISC-V.

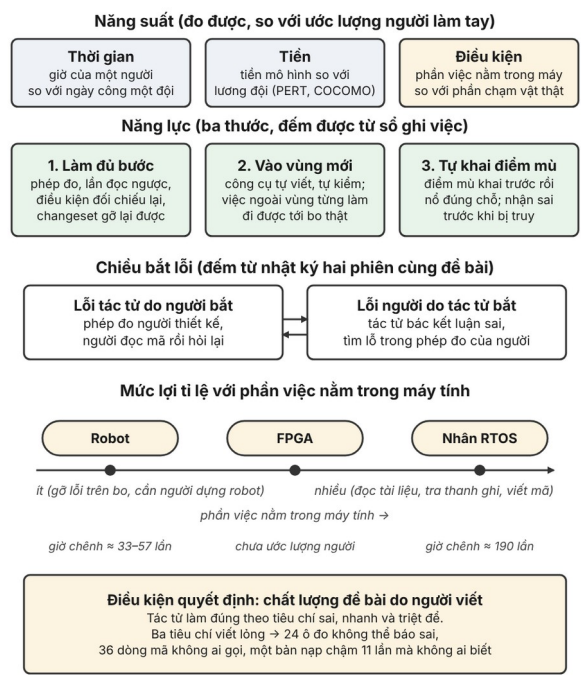
BẢNG II
BA VIỆC THẬT DÙNG LÀM DỮ LIỆU

	Nhân RTOS	Robot tự cân bằng	RISC-V trên FPGA
Bo	STM32F469I-DISCO, Cortex-M4F 180 MHz	ATmega328P, MPU6050, 2 × A4988	Tang Nano 20K, Gowin GW2AR-18
Đề bài	bỏ FreeRTOS, tự viết nhân tới khi LCD 800×480 và cảm ứng lên như bản cũ	từ hồ sơ phần cứng không có thuật toán, viết phần mềm để robot đứng	dựng SoC PicoRV32 [8], chạy C, đo nhân ma trận ở 3 cấu hình phần cứng
Mã tự viết	1 211 dòng (lần 2)	1 820 → 1 124 dòng	2 020 dòng
Lời gọi mô hình	351 → 837	1 415 → 670	1 307
Thời gian phiên	81,2 phút → 5,0 giờ	7,4 → 4,53 giờ	≈ 14,5 giờ
Kết quả trên bo	nhịp 1 003 Hz, 6 tác vụ, CFSR = 0	đứng được (cả hai lần)	96/96 ô, lệch 0 % với mô phỏng

Lần 2 của việc nhân RTOS dài hơn lần 1 vì lần 2 có bo thật: phần lớn thời gian thêm là đọc thanh ghi qua SWD khi bo tối, và vá một lỗi của chính EIDE. Việc FPGA cũng vậy: trong 14,5 giờ có bốn lỗi EIDE được tìm và vá, nên con số này không so được với một người đã có sẵn công cụ.

III. CÁCH ĐO

Hình 2 tóm tắt khung đo. Phần năng suất dùng hai thước quen thuộc (thời gian, tiền) kèm một điều kiện. Phần năng lực dùng ba thước mới, đếm được từ số ghi việc.



Hình 2. Khung đo: năng suất (hai thước, một điều kiện), năng lực (ba thước), chiều bắt lỗi, và yếu tố quyết định.

A. Năng suất

Cột tác tử là số đo: thời gian từ lượt đầu tới lượt cuối trong số ghi việc; tiền tính từ token mới, token đọc lại từ bộ đệm và token ra, theo ba kịch bản đơn giá. Cột người là ước lượng: một đội (1 Senior, 1 Mid, QA, quản lý) làm cùng đề bài, ước theo PERT ba mức cho 13 việc nhỏ, kiểm chéo bằng COCOMO làm mức trần. Hai cột không cùng loại số, và bài báo nói rõ điều đó ở mỗi chỗ dùng chúng.

B. Ba thước năng lực

Làm đủ bước: đếm trong số ghi việc số phép đo, số lần đọc ngược chip, số điều kiện được đổi chiều lại sau mỗi lần sửa, số changeset gỡ lại được. Đây là các việc kỹ sư làm tay biết là nên làm nhưng thường bỏ khi mới. *Vào vùng mới*: việc có nằm ngoài vùng tác tử từng có công cụ không, tác tử có tự viết công cụ để đi tiếp không, và kết quả có đi tới bo thật không. *Tự khai điểm mù*: số điểm yếu của bộ kiểm mà tác tử nêu ra trước khi bị hỏi, và trong đó bao nhiêu điểm sau đó nổ đúng chỗ; cộng số lần tác tử nhận sai trước khi bị truy.

C. Chiều bắt lỗi

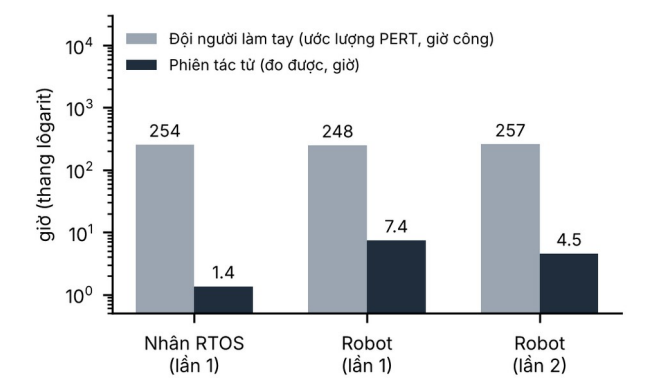
Với hai việc được làm hai lần, mỗi lỗi trong nhật ký được gán một chiều: lỗi của tác tử do người bắt (bằng một phép đo người thiết kế, hoặc người đọc mã rồi hỏi lại), hay lỗi của người do tác tử bắt (tác tử bác một kết luận của người, hoặc tìm ra lỗi trong phép đo người đề xuất). Lỗi của người được giữ nguyên trong nhật ký vì chúng đo được thứ mà lỗi của tác tử không đo: chất lượng đề bài.

IV. KẾT QUẢ

A. Năng suất: nhanh hơn, rẻ hơn, với bốn điều kiện

Hình 3 và Bảng III đặt giờ tác tử đo được cạnh giờ công ước lượng cho đội người. Với việc nhân RTOS, phiên tác tử lần 1 mất 81,2 phút so với 31,7 ngày công; với robot, 7,4 giờ rồi 4,53 giờ so với 31,0 và 32,1 ngày công. Về tiền, phiên robot lần 1 tốn khoảng 340 nghìn đồng tiền mô hình (mức

giữa trong ba kịch bản đơn giá), lần 2 khoảng 34 nghìn đồng, so với 105–112 triệu đồng lương ước lượng. Chênh khoảng 30–60 lần về giờ và 300–3 300 lần về tiền.



Hình 3. Giờ công ước lượng cho đội người (PERT, ngày công $\times$ 8) và giờ đo được của phiên tác tử, thang lôgarit. Việc FPGA không có cột người vì phần lớn thời gian phiên là vá chính EIDE.

BẢNG III
NĂNG SUẤT: TÁC TỬ (ĐO) VÀ ĐỘI NGƯỜI (ƯỚC LƯỢNG)

	Nhân RTOS (lần 1)	Robot (lần 1)	Robot (lần 2)
Giờ tác tử (đo)	81,2 phút	7,4 giờ	4,53 giờ
Ngày công đội (PERT)	31,7 $\pm$ 2,3	31,0 $\pm$ 2,6	32,1 $\pm$ 2,6
Tiền mô hình (mức giữa)	—	$\approx$ 340 nghìn đồng	$\approx$ 34 nghìn đồng
Lương đội (ước lượng)	$\approx$ 111 triệu đồng	$\approx$ 105 triệu đồng	$\approx$ 112 triệu đồng
Token đọc từ bộ đệm	—	93,6 %	90,8 %

Bốn điều các con số trên không chứng minh, nói trước để không bị đọc quá tay. (1) Người vẫn nằm trên đường quyết định: cả mười lần cắm bo của phiên robot lần 1 đều cần người dựng robot lên rồi nói nó ngã về phía nào; tác tử không chạm được vào robot, nên phần lớn 7,4 giờ là chờ người thử. (2) Bước cuối của lần 1 dùng một bản của nhà cung cấp đã chạy được để so; không có nó thì phiên còn kéo dài. (3) Tiền mô hình không phải toàn bộ chi phí: chưa tính giờ người, phần cứng và thời gian dựng EIDE. (4) Tám chỗ tác tử báo xong trong khi đang sai ở lần 1 đều cần một phép đo do người thiết kế; giao cùng việc cho người không biết đặt phép đo thì tám chỗ ấy lọt hết, và số tiền đó mua được một phần mềm trông như đã xong. Ngoài ra lần 2 rẻ hơn 10 lần không phải vì mô hình giỏi lên, mà vì phụ lục đề bài đã trao sẵn bộ tham số đã chạy thật, trong đó có hằng số hiệu chuẩn 92 mà lần 1 ghi sai thành 535 (điểm cân bằng lệch 3,1°).

B. Năng lực 1: làm đủ bước

Bảng IV đếm các việc mà một kỹ sư làm tay thường bỏ. Ở việc FPGA, 96 ô đo mỗi ô ba lượt lấy giá trị nhỏ nhất, mỗi ô một tổng kiểm đối chiếu với bốn giá trị người tính tay; 28 lần dựng bitstream, 80 lượt mô phỏng, mỗi lượt đọc lại kết quả. Ở việc RTOS, mức nước ngăn xếp của cả sáu tác vụ được quét định kỳ và ghi ra ô nhớ đọc được. Ở việc robot, 109 điều kiện bắt buộc dựng từ tài liệu được đối chiếu lại sau mỗi lần sửa, chip được đọc ngược đủ 32 lần trong một phiên, và 16 666 dòng số ghi việc kiểm được là chưa ai sửa dòng nào. Người

làm tay tới ô thứ ba mươi thường bắt đầu bỏ phần tổng kiểm; không phải tác tử cẩn thận hơn người, mà mỗi là lý do người bỏ bước, và nó không mới.

BẢNG IV
LÀM ĐỦ BƯỚC: ĐẾM TỪ SỐ GHI VIỆC

Việc tác tử làm đủ	Số đếm
Ô đo trên silicon, mỗi ô ba lượt và một tổng kiểm (FPGA)	96/96, 0 ô lệch tổng kiểm
Lần dựng bitstream / lượt mô phỏng (FPGA)	28 / 80
Điều kiện bắt buộc đối chiếu lại sau mỗi lần sửa (robot)	109
Lần đọc ngược chip trong một phiên (robot)	32
Mức nước ngăn xếp đo trên chip (RTOS)	6/6 tác vụ
Changeset gỡ lại được (RTOS / FPGA)	103 / 161
Dòng số ghi việc có móc bẫy (robot lần 1 / FPGA)	16 666 / 16 789

C. Năng lực 2: vào vùng mới

Việc FPGA bắt đầu từ chỗ EIDE không biết gì về HDL. Trong hai ngày, chuỗi công cụ mã mở được dựng (Yosys, nextpnr, Apicula, openFPGALoader, riscv64-unknown-elf-gcc), bốn công cụ EIDE được viết thêm (nạp FPGA, đọc mã chip, bắt bản ghi quanh lúc nạp, chốt phiên khi gói giao diện cũ hơn mã), và kết quả đi tới silicon: chuỗi UART in đúng mỗi 27 000 000 chu kỳ ở xung nhịp 27 MHz; 96/96 ô đo khớp bốn tổng kiểm tầng NGƯỜI; số chu kỳ trên bo lệch 0 so với mô phỏng; bộ nhân phần cứng nhanh hơn nhân bằng phần mềm 1,57–6,8 lần (bộ nhân tuần tự) và 2,45–12,5 lần (bộ nhân DSP); biến độc lập kiểm được trong mã máy (24 $\rightarrow$ 0 lời gọi `__mulsi3`). Tác tử còn trả lời được câu “lúc nào thêm phần cứng không giúp gì”: hai ô tăng ít nhất đều là số 8 bit với cách viết hoán vị vòng, nơi cấu hình không có bộ nhân đã nhanh sẵn (131 thay vì 502 chu kỳ cho một phép nhân-cộng), đúng theo định luật Amdahl.

Việc nhân RTOS cũng là vùng mới ở mức mã: tác tử viết đúng ngay phần khó nhất, chuyển ngữ cảnh bằng hợp ngữ qua PendSV, lập lịch và hàng đợi tĩnh, với 1 211 dòng tự viết bên cạnh 110 676 dòng của hãng không sửa, và 0 ký hiệu FreeRTOS còn trong ảnh. Khả năng cho phép đi vào vùng mới là tự viết công cụ rồi tự kiểm trước khi dùng: 8/8 ca trong bộ thử, ba công cụ trong phiên FreeRTOS trước đó, bốn công cụ cho FPGA.

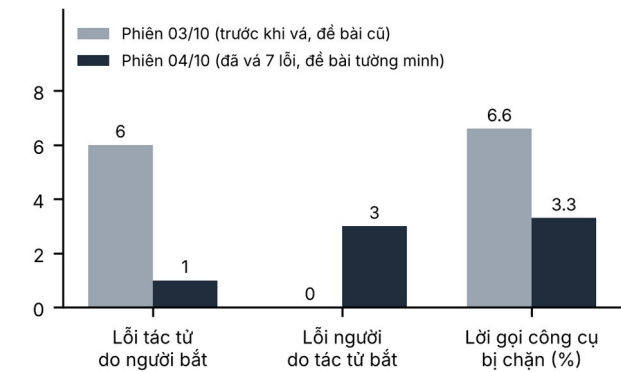
D. Năng lực 3: tự khai điểm mù

Ở việc RTOS, sau khi bộ kiểm xanh 4/4, tác tử tự phá mã bốn lần rồi báo hai ca bộ kiểm không bắt được: thứ tự xPSR và PC trên ngăn xếp, và giá trị EXC_RETURN, kèm dự đoán “ nạp lên bo sẽ nổ HardFault ngay chu kỳ đầu”. Người kiểm lại bằng tay: đổi EXC_RETURN thành 0, dịch lại, cả bốn ca vẫn xanh; khi chuyển ngữ cảnh chạy thật, bo nổ đúng IBUSERR ở đúng chỗ ấy. Một điểm mù khai trước thì khi nó nổ, biết ngay chỗ để tìm; nếu tác tử im, chỗ đó là một con bo tối với hàng chục nguyên nhân có thể. Ba lần khác tác tử nhận sai trước khi bị truy: khi được hỏi, nó nhận “đã phạm đúng lỗi này ở lượt

vừa rồi: chỉ viết hai hàm khung rỗng để dịch qua”; khi báo 38 tập .c mà tập thật có 42, nó nêu vì sao lệch; khi viết 36 dòng mã không ai gọi chỉ để một tiêu chí được thoả, nó ghi cả động cơ vào chú thích, nhờ đó 36 dòng ấy không lọt vào con số “tự viết”. Ở việc FPGA, hai lỗi tác tử tự tìm và tự báo đúng loại: lệnh chia bẫy CPU vì phân cứng để `ENABLE_DIV = 0`, và hàm memset tự viết bị trình dịch đổi thành lời gọi chính nó.

E. Chiều bắt lỗi đôi khi đề bài được viết lại

Phiên robot ngày 03/10 dừng chủ động ở bước 23/32 sau khi đo ra bảy chỗ tác tử sai; bảy chỗ ấy thành bảy bản vá cho EIDE, và đề bài được bổ sung phụ lục: mốc so sánh do người tự tính tay ghi ở tầng NGƯỜI, ba dòng nghiệm thu đòi đo tần số ngắt thật thay vì tin giá trị cấu hình, tham số đã chạy thật ghi kèm nguồn. Ngày 04/10 làm lại từ dự án trống. Hình 4 và Bảng V so hai phiên.



Hình 4. Hai phiên robot cùng đề bài: lỗi tác tử do người bắt giảm 6 → 1, lỗi người do tác tử bắt tăng 0 → 3, lỗi gọi công cụ bị chặn giảm 6,6 % → 3,3 %.

BẢNG V
HAI PHIÊN ROBOT, ĐẾM TỪ SỐ GHI VIỆC

Chỉ số	03/10	04/10
Lời gọi công cụ / bị chặn	394 / 26 (6,6 %)	541 / 18 (3,3 %)
Lỗi tác tử do người bắt	6 trong 7	1
Lỗi người do tác tử bắt	0	3
Ngắt phát xung đo thật	—	50,0005 kHz (+0,0009 %)
Dòng theo dõi thiếu / khoảng 100 ms đúng	—	0/369 / 372/372
Điều kiện phần chủ / phép phá mã bị bắt	—	22/22 / 4/4

Ba lỗi của người mà tác tử bắt được ở phiên 04/10: người kết luận điều kiện vòng 250 Hz đã đo xong bằng `millis()`; tác tử chỉ ra `millis()` phân giải 1 ms nên chu kỳ 4 ms có sai số tức thời tới $\pm 25\%$, phép đo chỉ chứng minh trung bình chứ không chứng minh các nhịp cách đều, và vòng lặp dùng bộ tích lũy triệt tiêu trôi dạt nên 25 nhịp ra đúng 100 ms do cách xây dựng kể cả khi các nhịp lệch nhau; nó cũng tìm ra lỗi đọc rách một biến 32 bit trên CPU 8 bit trong phép đo người đề xuất. Ở phiên 03/10 và ở lần 1, không một chỗ sai nào của tác tử do nó tự tìm ra. Chiều đối này không đến từ mô hình, vì mô hình không đối; nó đến từ bảy bản vá và từ đề bài được viết rõ ràng.

F. Tiêu chí viết lòng và hậu quả

Bảng VI ghi ba tiêu chí người viết lòng ở hai việc, hậu quả đo được, và cách đã sửa. Cả ba cùng một dạng: tiêu chí viết lòng thì không đo được gì, và tác tử làm đúng theo cái lòng ấy, nhanh và triệt để. Thứ cần soát trước khi soát mã của tác tử là câu mình vừa viết ra.

BẢNG VI
BA TIÊU CHÍ NGƯỜI VIẾT LÒNG, HẬU QUẢ VÀ CÁCH SỬA

Tiêu chí người viết	Hậu quả đo được	Đã sửa thành
“Mọi ô ok = 1” (FPGA): cờ ok chỉ so bốn cách viết với nhau	24/96 ô không thể báo sai; bo tính sai toàn bộ vẫn đạt	chốt luật sinh dữ liệu và bốn tổng kiểm người tính tay ở tầng NGƯỜI
“Mọi tệp mã nguồn đều vào được ảnh” (RTOS)	tác tử viết 36 dòng không ai gọi cho tệp rỗng, ghi cả động cơ vào chú thích	nêu trước danh sách tệp; tệp không có việc thì xóa
Thiếu tiêu chí “xung nhịp thật đạt 180 MHz” (RTOS)	bản nạp đầu chạy HSI 16 MHz, mọi mốc chậm 11,25 lần, không fault, không treo	do nhịp thật; giá trị cấu hình không phải phép đo

V. BÀN LUẬN

A. Mức lợi ti lệ với phần việc nằm trong máy tính

Hai việc có ước lượng người không chênh giống nhau: việc nhân RTOS chênh gấp hơn mười lần việc robot về tiền (khoảng 3 300 lần so với 300 lần) và gấp hơn ba lần về giờ (khoảng 190 lần so với 33–57 lần). Lý do là việc nhân RTOS gần như toàn bộ nằm trong phần đọc tài liệu, tra thanh ghi, viết mã, đúng phần tác tử nhanh; việc robot phần lớn thời gian nằm ở gỡ lỗi trên bo, mà phần đó tác tử không rút ngắn được bao nhiêu vì nó không chạm được vào robot. Việc nào càng dính vào vật thật thì khoảng chênh càng hẹp. Đây mới là quan sát trên hai việc, chưa phải quy luật; nhưng nó gợi ý cách chọn việc để giao cho tác tử.

B. “Mở rộng năng lực” nghĩa là gì với kỹ sư

Ba thước đo cho một hình dung cụ thể. Một kỹ sư làm tay với một con bo có thể viết được nhân thời gian thực, nhưng hiếm khi đo mức nước ngăn xếp của cả sáu tác vụ, đọc ngược chip 32 lần, hay đổi chiều 109 điều kiện sau mỗi lần sửa; tác tử làm các việc đó như một phần của mỗi lượt. Một kỹ sư chưa từng làm FPGA phải mất nhiều ngày dựng chuỗi công cụ trước khi viết dòng Verilog đầu tiên; tác tử dựng chuỗi công cụ, viết công cụ còn thiếu, và đi tới số đo trên silicon trong hai ngày, với điều kiện người cấp mốc so sánh độc lập. Và một bộ kiểm người viết hiếm khi kèm câu “bộ kiểm này không cạnh được hai chỗ sau, và đây là hậu quả nếu sai ở đó”. Năng lực mở rộng không nằm ở chỗ tác tử nghĩ hay hơn người, mà ở chỗ nó làm đủ, đi được vào chỗ chưa có đường, và nói ra chỗ nó không nhìn thấy.

C. Điều kiện để năng lực ấy có thật

Cùng dữ liệu cho thấy ba điều kiện. Thứ nhất, tác tử không tự biết nó sai: mọi câu “đã xong, đã kiểm, 0 byte lệch” đều có thể đúng về lời gọi và sai về kết quả, như lần `avrdude` báo `verified` với đúng tệp nó được đưa trong khi trên chip là bản khác. Thứ hai, bộ kiểm tác tử tự viết thiên về xanh: ba lần liên tiếp bộ kiểm chỉ chép logic sang tệp kiểm rồi so với chính nó, đúng hành vi [4] đo được; phải có người sửa hỏng mã rồi

đòi bộ kiểm báo lỗi. Thứ ba, nghiệm thu phải nằm ngoài tay tác tử: mốc do người tính trước, tiêu chí nêu trước khi chạy, và “robot đứng được” vẫn là quan sát của người. Hình dung đúng không phải tác tử thay người, mà là tác tử gánh phần ghi chép và phần làm đủ bước, người giữ phần đánh giá và phần chạm vào vật thật; và có một vai thứ ba hoá ra cũng quan trọng: một bản đã chạy được, dùng để so sánh.

D. Hạn chế

Ba việc, một người thao tác, một mô hình. Cột người là ước lượng của chính nhóm tác giả. Lần 2 của robot được trao sẵn tham số đã chạy thật. Việc FPGA chưa dừng lại được bằng make ngoài EIDE vì đường dẫn include tính từ gốc dự án; 96 ô là thật nhưng người khác gõ lại chưa ra. Tác tử chưa có công cụ chạy make hay chạy tệp Python nên hai lần phải nhờ người. Ba thước năng lực mới áp cho ba việc, chưa có mốc so sánh với kỹ sư làm tay cùng đề bài được đo thật.

VI. KẾT LUẬN

Trên ba việc nhúng thật, tác tử EIDE nhanh hơn và rẻ hơn ước lượng cho một đội người ở mức hàng chục lần về giờ và hàng trăm lần về tiền, nhưng con số đó chỉ đúng khi có người biết phải nghi chỗ nào. Điều bài báo muốn nói nằm ở ba thước đo kia: tác tử làm đủ 96 ô đo kèm tổng kiểm và 109 điều kiện đối chiếu lại; đi từ không có công cụ HDL tới số đo trên silicon trong hai ngày; và tự khai hai điểm mù rồi nổ đúng chỗ. Đó là những việc kỹ sư làm tay thường bỏ hoặc chưa làm được, nên tác tử mở rộng năng lực chứ không chỉ rút ngắn thời gian. Hai điều kiện đi kèm: mức lợi tỉ lệ với phần việc nằm trong máy tính, và chất lượng đề bài do người viết quyết định kết quả, thể hiện qua chiều bắt lỗi đối hân khi đề bài được viết lại cho rõ. Hướng tiếp theo là đo ba thước này với nhiều người thao tác và nhiều mô hình, và dựng mốc kỹ sư làm tay cùng đề bài để thay cột ước lượng bằng số đo.

TÀI LIỆU THAM KHẢO

- [1] Thủ tướng Chính phủ, “Quyết định số 21/2026/QĐ-TTg ban hành Danh mục công nghệ chiến lược và Danh mục sản phẩm công nghệ chiến lược,” 30/4/2026. [Trực tuyến]. <https://congbao.chinhphu.vn/van-ban/quyet-dinh-so-21-2026-qd-ttg-469478.htm>
- [2] Z. Englhardt, R. Li, D. Nissanka, Z. Zhang, G. Narayanswamy, J. Breda, X. Liu, S. Patel, và V. Iyer, “Exploring and characterizing large language models for embedded system development and debugging,” trong Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, ACM, 2024. arXiv:2307.03817.
- [3] Y. Li và cộng sự, “Skilled AI agents for embedded and IoT systems development,” arXiv:2603.19583, 2026.
- [4] Z. Zhong, A. Raghunathan, và N. Carlini, “ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases,” arXiv:2510.20270, 2025.
- [5] Vũ Trí Công và Nguyễn Trung Hiếu, “Kiến trúc tác tử hỗ trợ lập trình nhúng: từ định tuyến ý định đến vòng lặp có kiểm soát,” bản thảo gửi Hội nghị REV-ECIT 2026.
- [6] Vũ Trí Công và Nguyễn Trung Hiếu, “Ứng dụng tác tử AI trong lập trình nhúng: kiến trúc, những việc làm được và kết quả trên ba bo mạch thật,” bản thảo gửi Hội nghị REV-ECIT 2026.
- [7] Vũ Trí Công và Nguyễn Trung Hiếu, “EIDE v3: lỗi tác tử kỹ sư nhúng; mã nguồn, số ghi việc và nhật ký ba việc thật,” 2026. [Trực tuyến]. <https://github.com/mobiluckvn/EIDEv3>

- [8] C. Wolf, “PicoRV32: a size-optimized RISC-V CPU,” mã nguồn mở. [Trực tuyến]. <https://github.com/YosysHQ/picorv32>